

N2_Probabilitat_i_estadistica_basica

November 26, 2025

1 Probabilitat i estadística básica

Disposen del fitxer `alumnes_dades_correlacionades.csv` a partir del qual explicarem alguns conceptes bàsics de probabilitat i estadística de manera visual i sense fórmules matemàtiques.

El contingut del fitxer és el següent:

- **ID_Alumne:** Identificador únic de l'alumne (format com "ALUM-XXXX").
- **Nom_Complet:** Nom i cognom de l'alumne.
- **Edat:** Edat de l'alumne.
- **Alçada:** Alçada de l'alumne en metres.
- **Hores d'Estudi:** Nombre d'hores treballades per l'alumne fora de l'aula.
- **Faltes d'Assistència:** Nombre de faltes d'assistència.
- **Programació, Bases de Dades, Desenvolupament Web, Sistemes Operatius:** Notes obtingudes en aquestes assignatures.

```
[ ]: !wget https://raw.githubusercontent.com/I-Onna/dades/refs/heads/main/
      ↪alumnes_dades_correlacionades.csv
```

```
--2025-11-21 15:51:09-- https://raw.githubusercontent.com/I-
Onna/dades/refs/heads/main/alumnes_dades_correlacionades.csv
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13584 (13K) [text/plain]
Saving to: 'alumnes_dades_correlacionades.csv'

alumnes_dades_corre 100%[=====>] 13.27K --.-KB/s in 0.003s

2025-11-21 15:51:09 (4.13 MB/s) - 'alumnes_dades_correlacionades.csv' saved
[13584/13584]
```

1.0.1 Exploració de les Dades

```
[ ]:
```

El primer pas per fer-nos una primera idea del contingut.

```
[ ]: import pandas as pd
```

```
df = pd.read_csv('alumnes_dades_correlacionades.csv')
df.head()
```

```
[ ]: ID Alumne      Nom  Edat  Alçada (m)  Programació \
0  ALU-1000    Efraín Cifuentes    19      1.69      5.3
1  ALU-1001      Leyre Goñi    44      1.59      6.9
2  ALU-1002    Martin Palacios    23      1.70      7.4
3  ALU-1003      Pastor Reyes    22      1.76      7.0
4  ALU-1004  Vicente Montesinos    18      1.74      4.9

      Bases de Dades  Sistemes Operatius  Desenvolupament Web  Hores d'Estudi \
0              5.8              10.0              6.9              14
1              6.2              3.1              8.5              10
2              4.1              6.4              8.0              5
3              8.4              3.7             10.0             20
4              0.2              6.0              4.4             17

      Faltes d'Assistència
0              4
1              8
2             10
3              0
4             12
```

```
[ ]: files, columnes = df.shape

print(f"Té {files} files (registres) i {columnes} columnes (atributs).")
```

Té 250 files (registres) i 10 columnes (atributs).

```
[ ]: n_elements = df.size
print(f"Té un total de {n_elements} elements.")
```

Té un total de 2500 elements.

```
[ ]: df.describe()
```

```
[ ]: count      Edat  Alçada (m)  Programació  Bases de Dades  Sistemes Operatius \
count    250.00000  250.000000  250.000000    250.000000    250.000000
mean      22.18000    1.724120    6.918000     5.153600     7.102800
std        5.55039    0.053536    1.021317     2.721565     2.941032
min       18.00000    1.590000    4.300000     0.000000     0.000000
25%       19.00000    1.690000    6.200000     3.300000     5.100000
50%       21.00000    1.720000    6.900000     5.250000     7.750000
75%       23.75000    1.760000    7.600000     7.100000    10.000000
max       44.00000    1.930000   10.100000    10.000000    10.000000
```

	Desenvolupament Web	Hores d'Estudi	Faltes d'Assistència
count	250.0000	250.000000	250.000000
mean	6.7852	14.276000	6.220000
std	2.1311	7.022041	8.238147
min	0.0000	0.000000	0.000000
25%	5.5250	9.250000	0.000000
50%	6.8500	14.000000	1.000000
75%	8.2750	19.000000	11.000000
max	10.0000	30.000000	25.000000

Teniu una “descripció” de les dades, però realment què volen dir tots aquests números?

1.0.2 Descripció de les Dades

Utilitzarem diversos termes estadístics que ens ajudaran resumir i a tenir una idea de les característiques d'un conjunt de dades de forma ràpida i efectiva.

Ens ajuden a descriure els següents aspectes del conjunt de dades:

- Tendència Central (Mitjana, Mediana)
- Dispersió (Variància, Desviació Estàndard, Rang)
- Forma de la Distribució (Asimetria, Curtosi)
- Distribució (Histograma, Percentils, Quartils)
- Valors Extrems (Outliers)

Mitjana i Mediana La mitjana i la mediana són paràmetres estadístics que ens ajuden a determinar el valor central d'un conjunt de dades.

- **Mitjana:** és el valor obtingut sumant tots els valors d'un conjunt de dades i dividint el resultat pel nombre total de valors. És el “promig” dels valors.
- **Mediana:** és el valor que es troba al mig d'un conjunt de dades quan estan ordenades en ordre. Divideix les dades en dues meitats iguals (amb el mateix nombre de dades).

Primer de tot, un exemple il·lustratiu del concepte de mitjana i mediana:

```
[ ]: import numpy as np
import matplotlib.pyplot as plt

dades = np.array([1, 2, 2, 3, 4, 5, 5, 6, 8, 20])

mitjana = np.mean(dades)
mediana = np.median(dades)

plt.plot(dades, 'go', label='Dades', markersize=10)
plt.axvline(x=mitjana, color='red', linestyle='--', label=f'Mitjana: {mitjana:.2f}')
plt.axvline(x=mediana, color='blue', linestyle='--', label=f'Mediana: {mediana:.2f}')
```

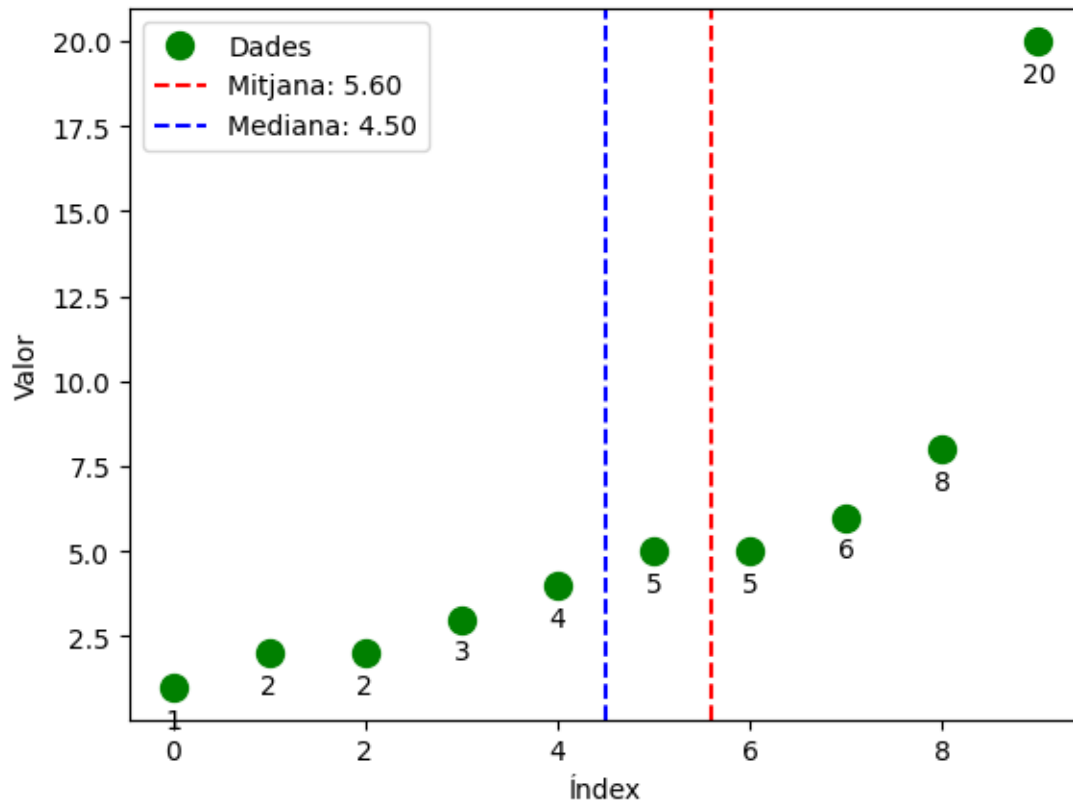
```

for i, valor in enumerate(dades):
    plt.annotate(str(valor), (i, valor), textcoords="offset points",
        ↪xytext=(0,-15), ha='center')

plt.xlabel('Índex')
plt.ylabel('Valor')

plt.legend()
plt.show()

```



Ara seguim amb els càlculs amb les dades del fitxer alumnes_dades_correlacionades.csv

```

[ ]: # Calculem la mitjana d'hores treballades
mean_hores = df['Hores d'\Estudi'].mean()
print(mean_hores)
print(type(mean_hores))
print("Mitjana d'hores treballades pels alumnes: ", mean_hores.round(2))

```

14.276

<class 'numpy.float64'>

Mitjana d'hores treballades pels alumnes: 14.28

```
[ ]: # Calculem la mediana d'hores treballades
mediana_hores = df['Hores d\'Estudi'].median()
print(mediana_hores)
print(type(mediana_hores))
print("Mediana d'hores treballades pels alumnes: ", mediana_hores)
```

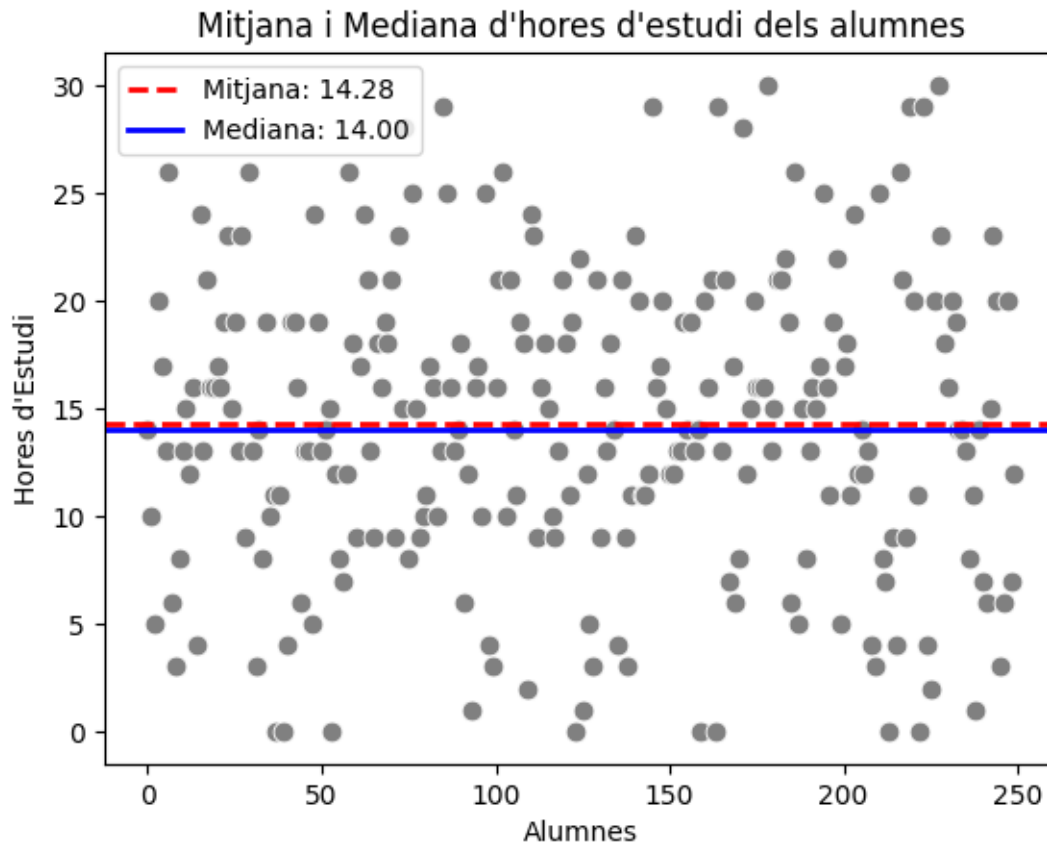
```
14.0
<class 'float'>
Mediana d'hores treballades pels alumnes: 14.0
```

```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

files, columnes = df.shape
sns.scatterplot(x=range(files), y=df['Hores d\'Estudi'], color='grey', s=60)

# Afegir línies per a la mitjana i la mediana
plt.axhline(df['Hores d\'Estudi'].mean(), color='red', linestyle='--',
            linewidth=2, label=f"Mitjana: {df['Hores d\'Estudi'].mean():.2f}")
plt.axhline(df['Hores d\'Estudi'].median(), color='blue', linestyle='-',
            linewidth=2, label=f"Mediana: {df['Hores d\'Estudi'].median():.2f}")

plt.xlabel('Alumnes')
plt.ylabel('Hores d\'Estudi')
plt.title('Mitjana i Mediana d\'hores d\'estudi dels alumnes')
plt.legend()
plt.show()
```



La mitjana i la mediana representen conceptes similars però ens poden donar informació sobre la distribució de les dades.

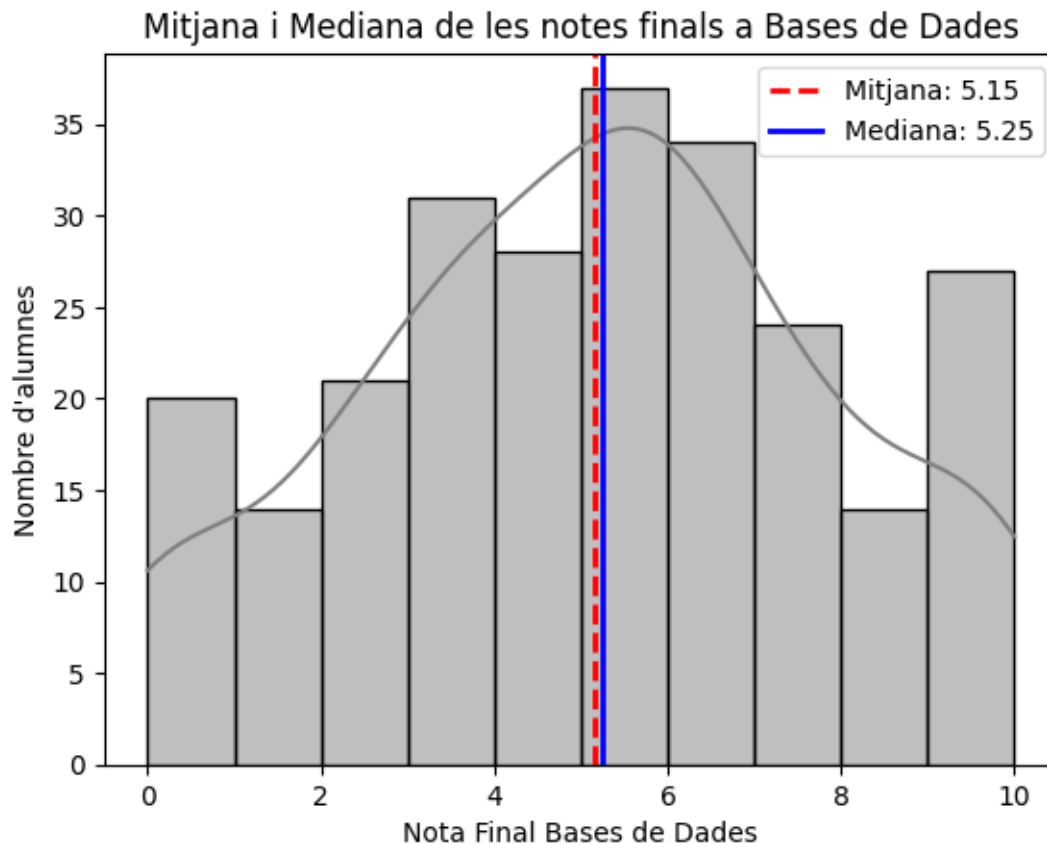
Si els seus valors són molt propers ens indica que: * **Les dades tenen una distribució simètrica:** tenim aproximadament el mateix nombre de dades per sota que per sobre del valor central. * **No comptem amb la presència d'outlayers (valors atípics):** no existeixen valors extremadament alts o baixos. * **Les dades poden molt probablement seguir una distribució normal (o gaussiana):** en forma de “campana de Gauss”.

```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

# Histograma amb la mitjana i la mediana
sns.histplot(df['Bases de Dades'], bins=10, kde=True, color='grey',
             edgecolor='black')
plt.axvline(df['Bases de Dades'].mean(), color='red', linestyle='--',
            linewidth=2, label=f"Mitjana: {df['Bases de Dades'].mean():.2f}")

plt.axvline(df['Bases de Dades'].median(), color='blue', linestyle='-',
            linewidth=2, label=f"Mediana: {df['Bases de Dades'].median():.2f}")
```

```
plt.xlabel('Nota Final Bases de Dades')
plt.ylabel('Nombre d\'alumnes')
plt.title('Mitjana i Mediana de les notes finals a Bases de Dades')
plt.legend()
plt.show()
```



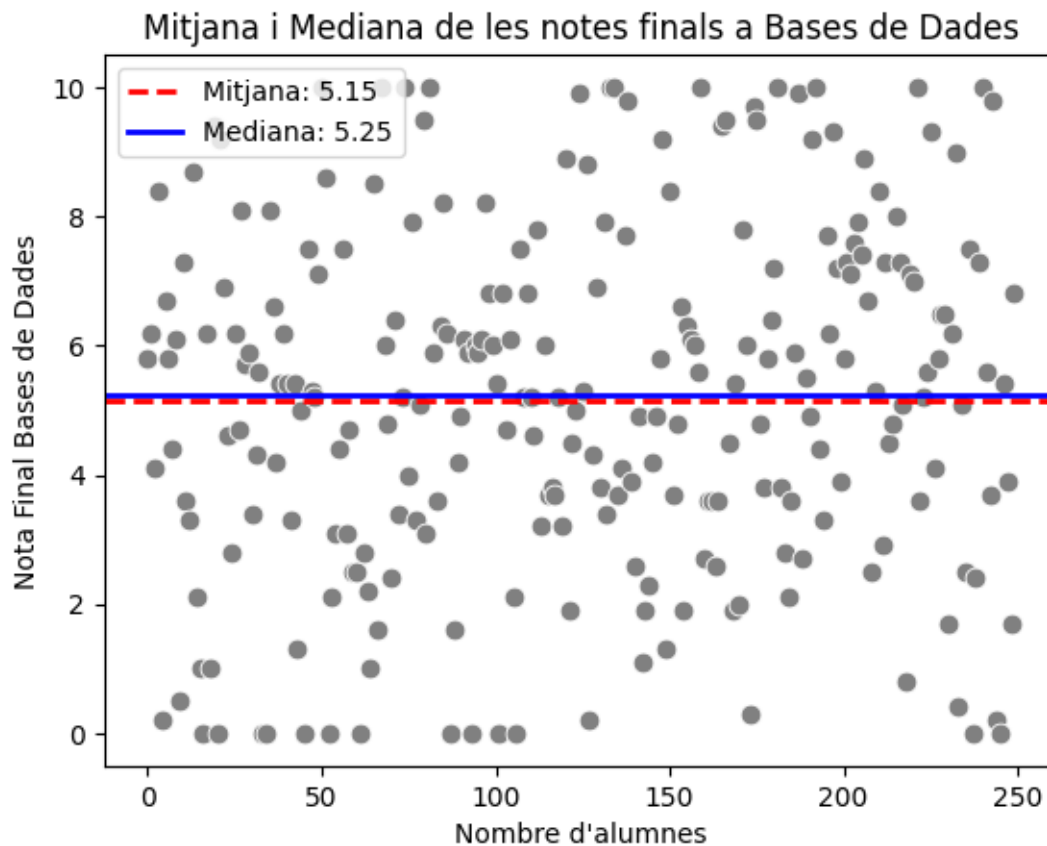
```
[ ]: df['Bases de Dades'].mean()
df['Bases de Dades'].median()
```

```
[ ]: 5.25
```

```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

files, columns = df.shape
sns.scatterplot(x=range(files), y=df['Bases de Dades'], color='grey', s=60)
# Afegir línies per a la mitjana i la mediana
plt.axhline(df['Bases de Dades'].mean(), color='red', linestyle='--',
            linewidth=2, label=f"Mitjana: {df['Bases de Dades'].mean():.2f}")
```

```
plt.axhline(df['Bases de Dades'].median(), color='blue', linestyle='-',  
            linewidth=2, label=f"Mediana: {df['Bases de Dades'].median():.2f}")  
plt.ylabel('Nota Final Bases de Dades')  
plt.xlabel('Nombre d\'alumnes')  
plt.title('Mitjana i Mediana de les notes finals a Bases de Dades')  
plt.legend()  
plt.show()
```

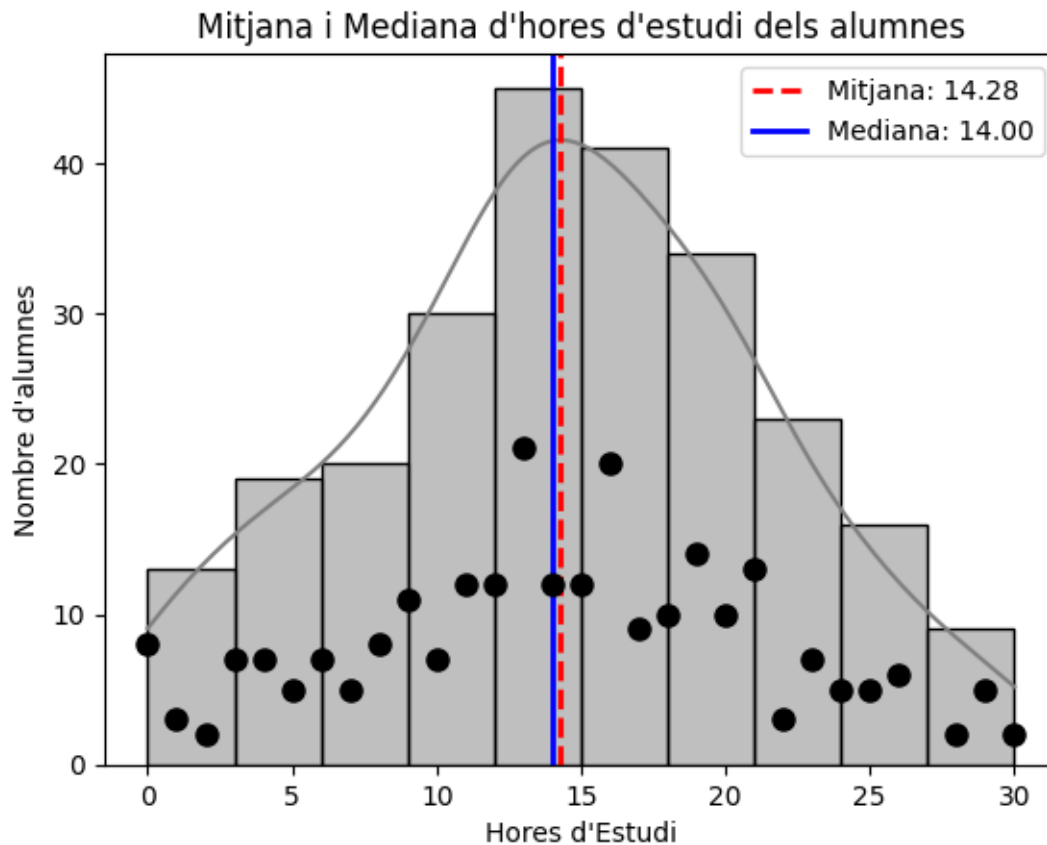


```
[ ]: import seaborn as sns  
import matplotlib.pyplot as plt  
  
# Histograma amb la mitjana i la mediana  
sns.histplot(df['Hores d\'Estudi'], bins=10, kde=True, color='grey',  
             edgecolor='black')  
plt.axvline(df['Hores d\'Estudi'].mean(), color='red', linestyle='--',  
            linewidth=2, label=f"Mitjana: {df['Hores d\'Estudi'].mean():.2f}")  
plt.axvline(df['Hores d\'Estudi'].median(), color='blue', linestyle='-',  
            linewidth=2, label=f"Mediana: {df['Hores d\'Estudi'].median():.2f}")  
count = df['Hores d\'Estudi'].value_counts()
```



```
plt.plot(count.index, count.values, marker='o', color='black', linestyle='',
↪markersize=8)

plt.xlabel('Hores d\'Estudi')
plt.ylabel('Nombre d\'alumnes')
plt.title('Mitjana i Mediana d\'hores d\'estudi dels alumnes')
plt.legend()
plt.show()
```



Simulem un seguit d'alumnes *outlayers*:

```
[ ]: files, columns = df.shape

df_copy = df.copy() #per poder utilitzar les dades en altres exemple
# Assignem 30 hores a les últimes 20 files de la columna 'Hores Estudi' ->
↪creem un outlayer
df_copy.loc[files - 30:, 'Hores d\'Estudi'] = 20

dades_outlayers = df_copy
dades_outlayers.tail()
```

```
[ ]:      ID Alumne      Nom  Edat  Alçada (m)  Programació  \
245  ALU-1245      Paco Fortuny   32      1.73      7.5
246  ALU-1246  Lorenza Arellano   20      1.80      5.8
247  ALU-1247      Josué Conesa   36      1.68      7.7
248  ALU-1248  Rodrigo Quintana   36      1.76      6.8
249  ALU-1249  Benjamín Palomo   25      1.64      6.6

      Bases de Dades  Sistemes Operatius  Desenvolupament Web  Hores d'Estudi  \
245      0.0      3.3      6.1      20
246      5.4      5.5      3.7      20
247      3.9     10.0      9.2      20
248      1.7      5.4      4.6      20
249      6.8     10.0      7.4      20

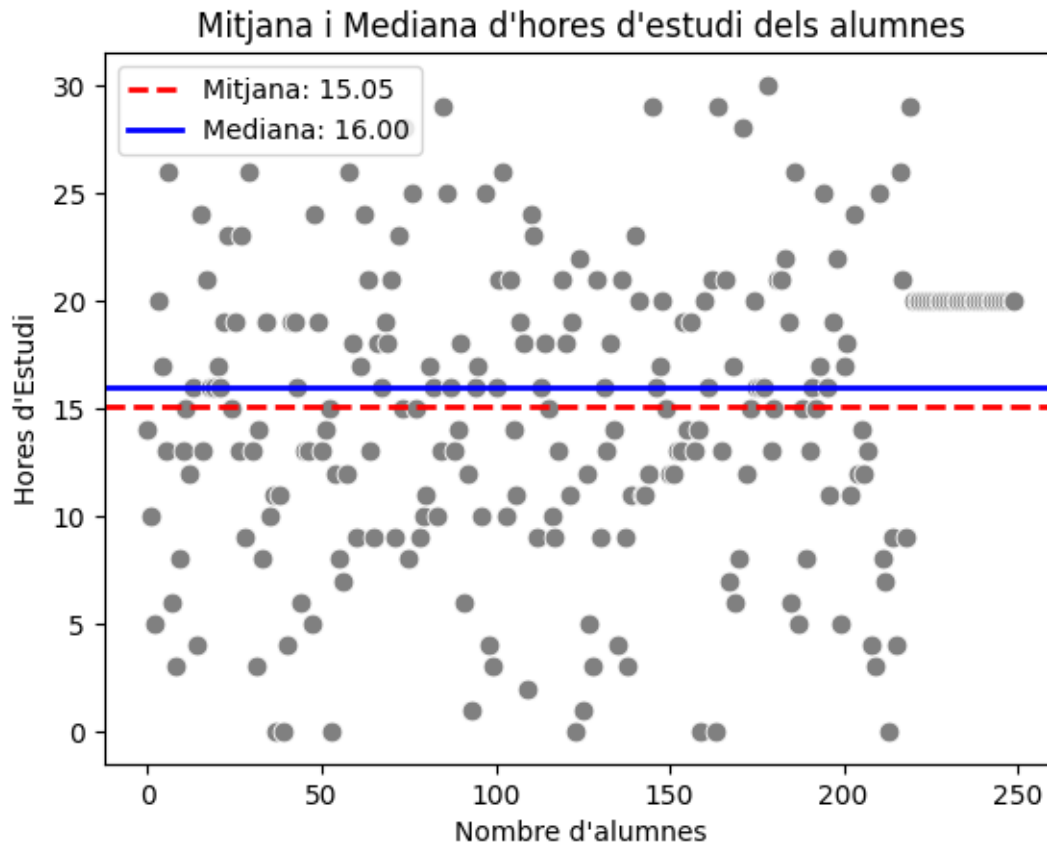
      Faltes d'Assistència
245      4
246     25
247      0
248     12
249     10
```

```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

files, columnes = dades_outlayers.shape
print(files)
sns.scatterplot(x=range(files), y=dades_outlayers['Hores d\'Estudi'],
               color='grey', s=60)

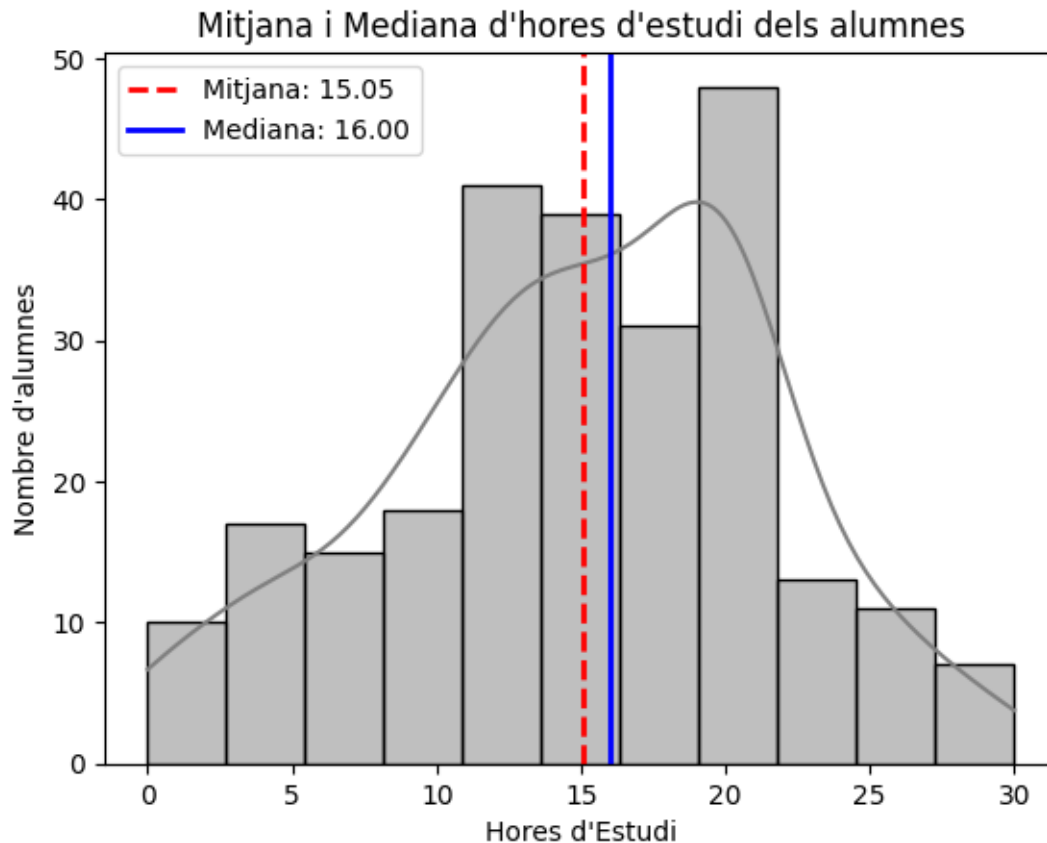
# Afegim les línies per a la mitjana i la mediana
plt.axhline(dades_outlayers['Hores d\'Estudi'].mean(), color='red',
            linestyle='--', linewidth=2, label=f"Mitjana: {dades_outlayers['Hores d\'Estudi'].mean():.2f}")
plt.axhline(dades_outlayers['Hores d\'Estudi'].median(), color='blue',
            linestyle='-', linewidth=2, label=f"Mediana: {dades_outlayers['Hores d\'Estudi'].median():.2f}")
plt.ylabel('Hores d\'Estudi')
plt.xlabel('Nombre d\'alumnes')
plt.title('Mitjana i Mediana d\'hores d\'estudi dels alumnes')
plt.legend()
plt.show()
```

250



```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

# Histograma amb la mitjana i la mediana
sns.histplot(dades_outlayers['Hores d\'Estudi'], bins=11, kde=True,
             color='grey', edgecolor='black')
plt.axvline(dades_outlayers['Hores d\'Estudi'].mean(), color='red',
            linestyle='--', linewidth=2, label=f"Mitjana: {dades_outlayers['Hores d\'Estudi'].mean():.2f}")
plt.axvline(dades_outlayers['Hores d\'Estudi'].median(), color='blue',
            linestyle='-', linewidth=2, label=f"Mediana: {dades_outlayers['Hores d\'Estudi'].median():.2f}")
plt.xlabel('Hores d\'Estudi')
plt.ylabel('Nombre d\'alumnes')
plt.title('Mitjana i Mediana d\'hores d\'estudi dels alumnes')
plt.legend()
plt.show()
```



Mode

En estadística, la moda és la dada o valor que apareix amb més freqüència dins d'una sèrie de dades. És una mesura de tendència central que indica quin valor és el més comú.

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt

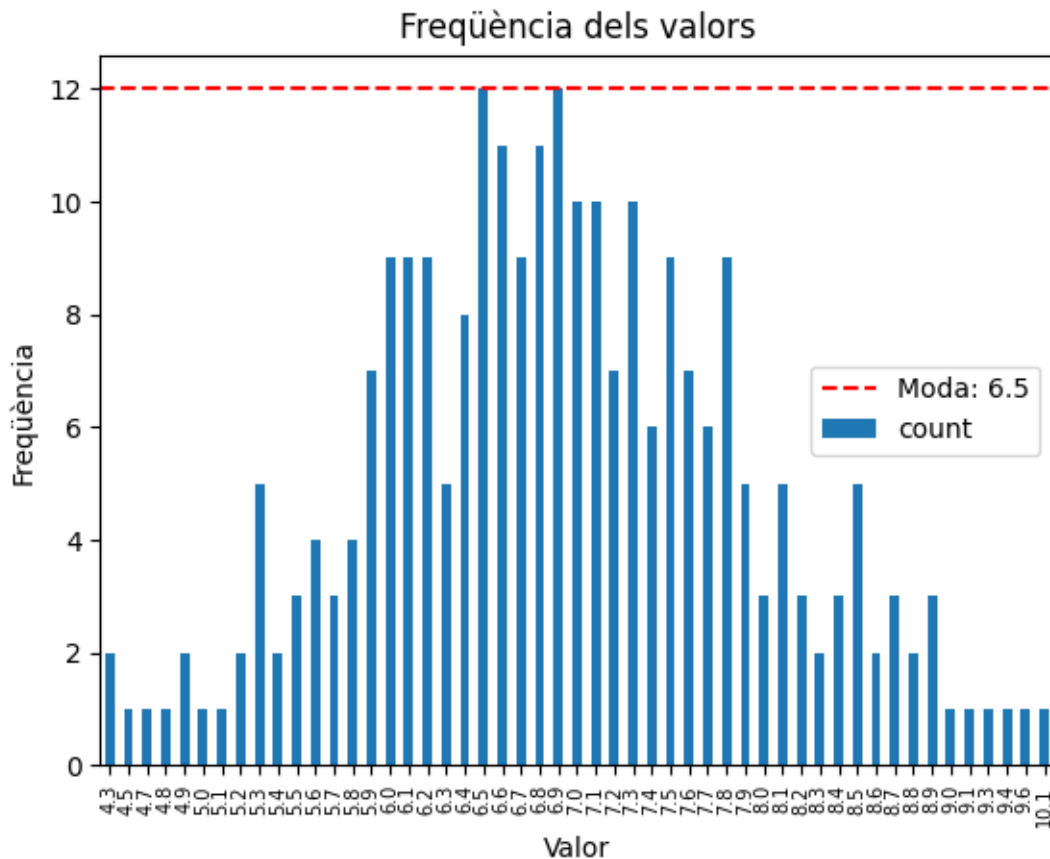
# Calculem la moda
moda = df['Programació'].mode()[0]

# Mostrem la moda
print(f"La moda és: {moda}")

# Visualitzem les freqüències
df['Programació'].value_counts().sort_index().plot(kind='bar')
plt.title("Freqüència dels valors")
plt.xlabel("Valor")
plt.ylabel("Freqüència")
```

```
plt.axhline(y=df['Programació'].value_counts()[moda], color='r',
            linestyle='--', label=f'Moda: {moda}')
plt.xticks(rotation=90, fontsize=7)
plt.legend()
plt.show()
```

La moda és: 6.5



```
[ ]: df['Programació'].mode().head()
```

```
[ ]: 0    6.5
      1    6.9
      Name: Programació, dtype: float64
```

Variància, Desviació Estàndard i Rang: Són mesures que ens ajuden a entendre com es distribueixen els valors dins d'un conjunt de dades. Aquestes mesures ens donen informació sobre la “dispersió” dels valors, és a dir, si els valors són molt diferents entre si o si són bastant similars.

- **Variància:** mesura que ens indica la desviació del conjunt de valors respecte a la mitjana. Indica la dispersió de la dades.

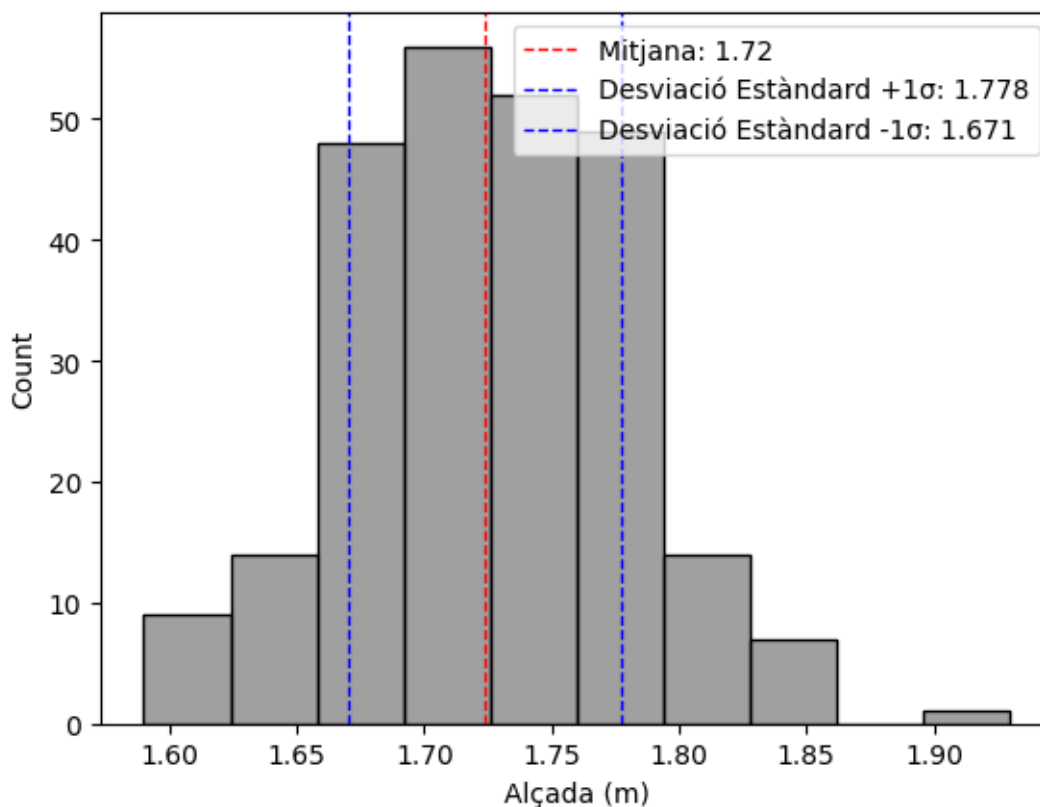
- **Desviació estàndard:** es calcula a partir de la variança i mesura la variabilitat dels valors al voltant de la mitjana en les mateixes unitats que les dades.
- **Rang:** diferència entre el valor màxim i mínim del conjunt de dades. Indica l'interval total cobert pels valors.

```
[ ]: # Calculem variància, desviació estàndard i rang
import seaborn as sns
import matplotlib.pyplot as plt

variancia = df['Alçada (m)'].var()
mitjana = df['Alçada (m)'].mean()
desviacio_estandard = df['Alçada (m)'].std()
rang = df['Alçada (m)'].max() - df['Alçada (m)'].min()

sns.histplot(df['Alçada (m)'], bins=10, edgecolor='black', color='grey')
plt.axvline(mitjana, color='red', linestyle='dashed', linewidth=1,
            label=f'Mitjana: {mitjana:.2f}')

# Afegim la representació d'errors per a la desviació estàndard
plt.axvline(mitjana + desviacio_estandard, color='blue', linestyle='dashed',
            linewidth=1, label=f'Desviació Estàndard +1: {mitjana + desviacio_estandard:.3f}')
plt.axvline(mitjana - desviacio_estandard, color='blue', linestyle='dashed',
            linewidth=1, label=f'Desviació Estàndard -1: {mitjana - desviacio_estandard:.3f}')
plt.legend()
plt.show()
```

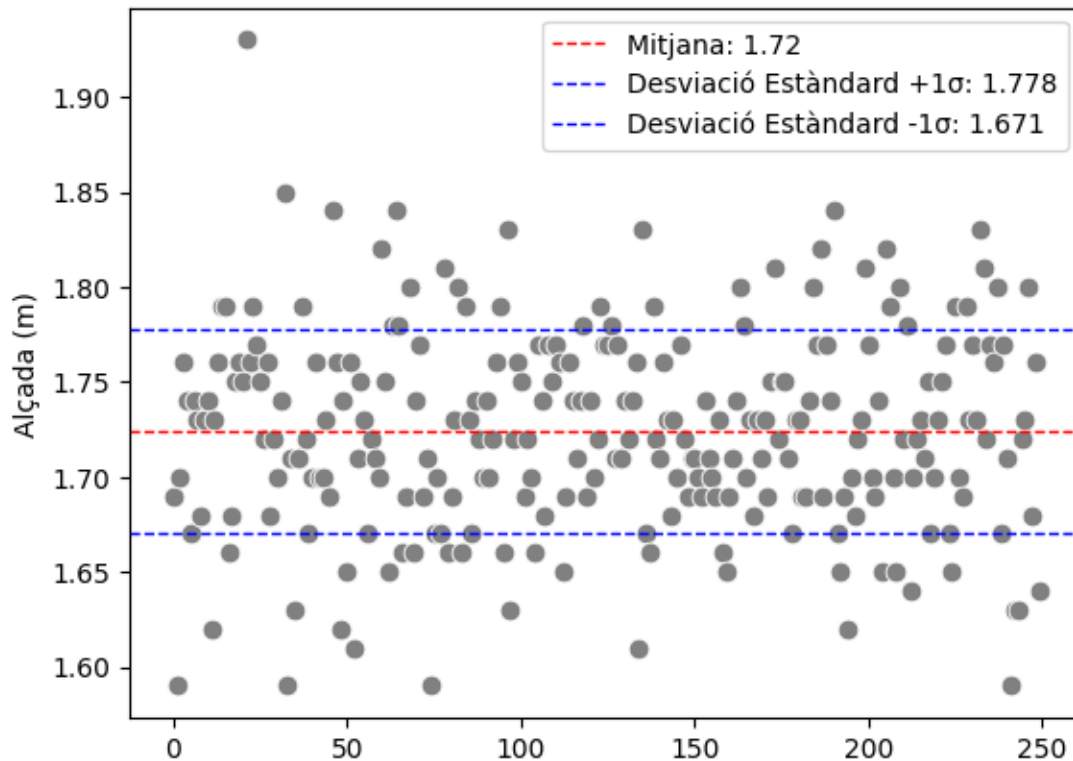


```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

files, columnes = df.shape
sns.scatterplot(x=range(files), y=dades_outlayers['Alçada (m)'], color='grey',
               ↪s=60)

plt.axhline(mitjana, color='red', linestyle='dashed', linewidth=1,
           ↪label=f'Mitjana: {mitjana:.2f}')

# Afegim la representació d'errors per a la desviació estàndard
plt.axhline(mitjana + desviacio_estandard, color='blue', linestyle='dashed',
           ↪linewidth=1, label=f'Desviació Estàndard +1: {mitjana + desviacio_estandard:.
           ↪3f}')
plt.axhline(mitjana - desviacio_estandard, color='blue', linestyle='dashed',
           ↪linewidth=1, label=f'Desviació Estàndard -1: {mitjana - desviacio_estandard:.
           ↪3f}')
plt.legend()
plt.show()
```



```
[ ]: print(f"Mitjana: {mitjana:.3f}")
      print(f"Variança: {variancia:.3f}")
      print(f"Desviació Estàndard: {desviacio_estandard:.3f}")
      print(f"Rang: {rang:.3f}")
```

Mitjana: 1.724

Variança: 0.003

Desviació Estàndard: 0.054

Rang: 0.340

Modifiquem la columna alçada per modificar-ne la variança.

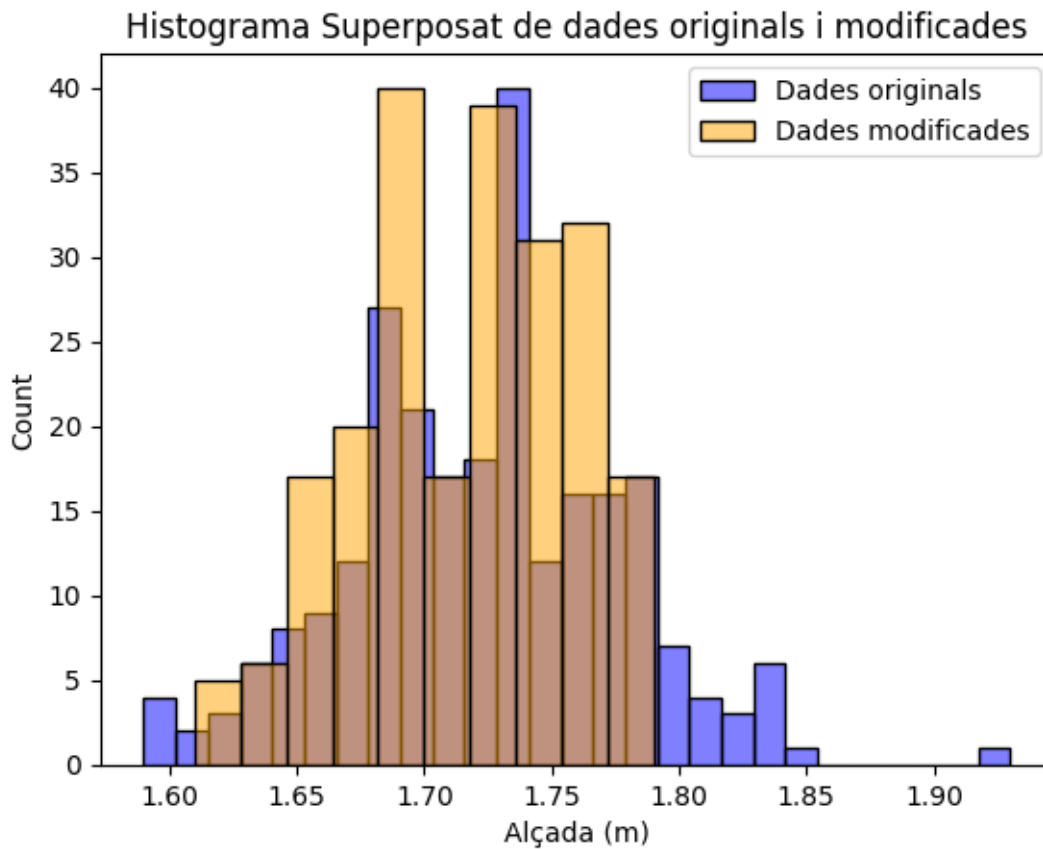
```
[ ]: df_copy = df.copy() #necessari per recuperar no modificar les dades inicial
      df_modificada = df_copy[(df_copy['Alçada (m)'] > 1.6) & (df_copy['Alçada (m)']_
      ↪ < 1.8)]
```

```
[ ]: # Crear els histogrames superposats per facilitar la comparació
      sns.histplot(df['Alçada (m)'], bins=27, kde=False, color='blue', label='Dades_
      ↪ originals', alpha=0.5)
      sns.histplot(df_modificada['Alçada (m)'], bins=10, kde=False, color='orange',_
      ↪ label='Dades modificades', alpha=0.5)
```

```
# Afegir títol i etiquetes
```



```
plt.title('Histograma Superposat de dades originals i modificades')
plt.xlabel('Alçada (m)')
plt.ylabel('Count')
plt.legend()
plt.show()
```



Normalitzant les dades podrem veure la forma relativa de cada distribució independentment del nombre total de registres de cada conjunt de dades.

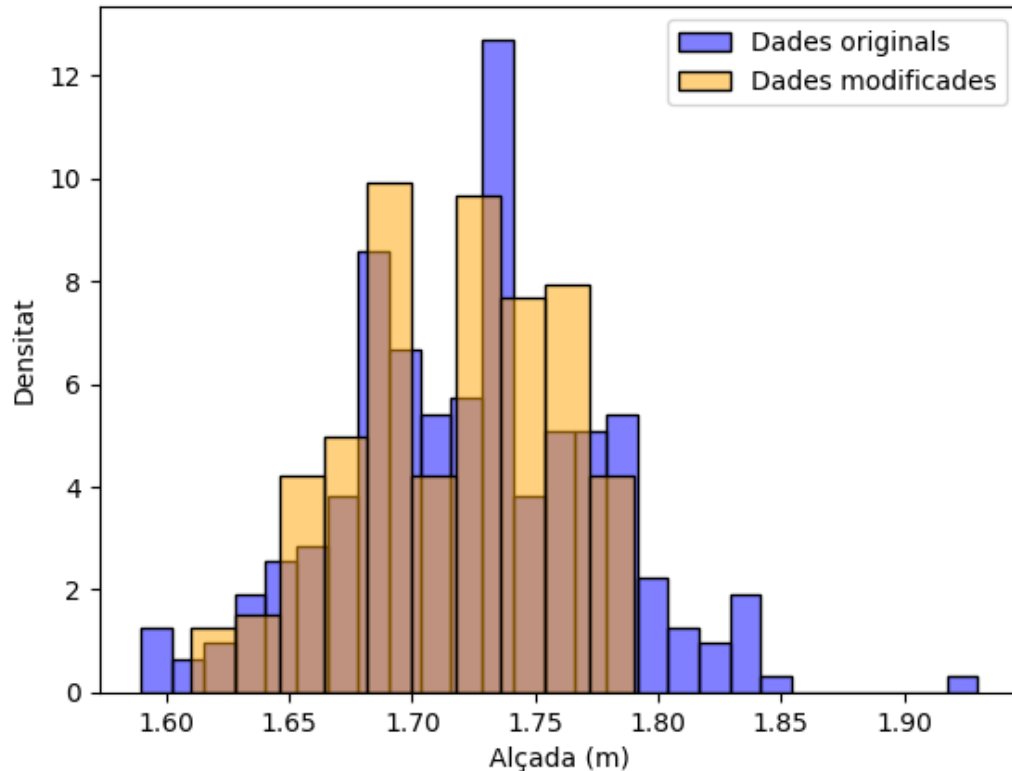
```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns

# Crear els histogrames normalitzats superposats
sns.histplot(df['Alçada (m)'], bins=27, kde=False, color='blue', label='Dades originals', alpha=0.5, stat='density')
sns.histplot(df_modificada['Alçada (m)'], bins=10, kde=False, color='orange', label='Dades modificades', alpha=0.5, stat='density')

# Afegir títol i etiquetes
plt.title('Histograma Superposat de Dades Originals i Modificades Normalitzats')
```

```
plt.xlabel('Alçada (m)')
plt.ylabel('Densitat')
plt.legend()
plt.show()
```

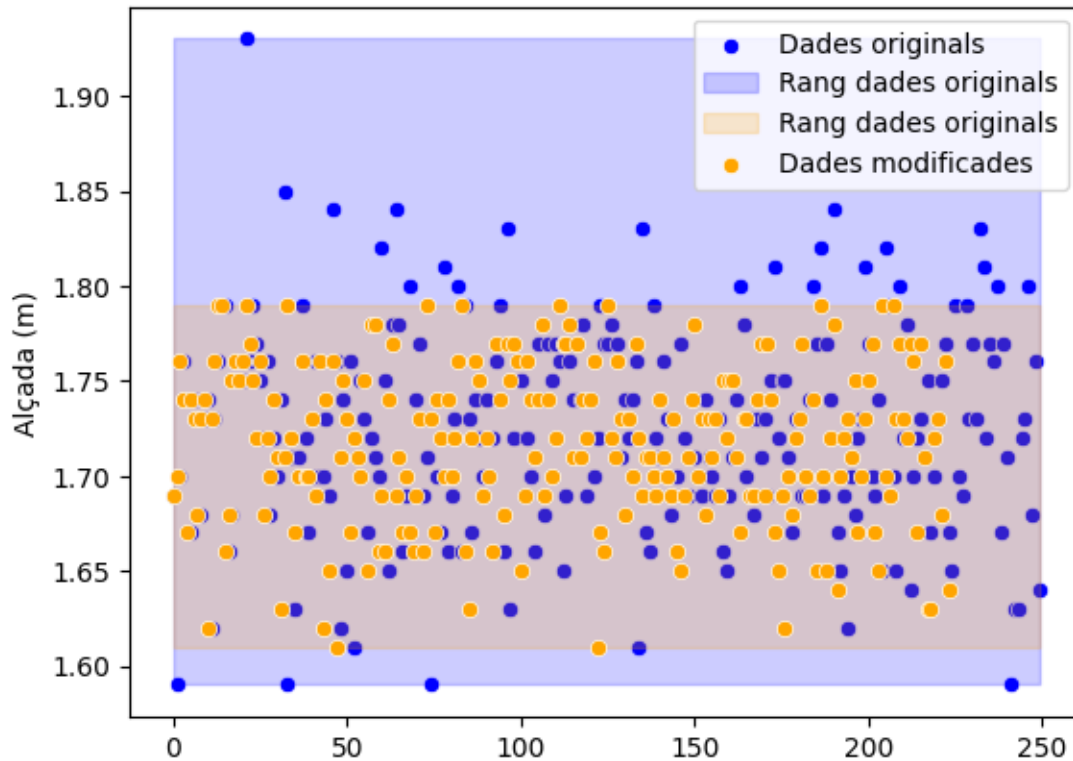
Histograma Superposat de Dades Originals i Modificades Normalitzats



```
[ ]: files, columnes = df.shape
sns.scatterplot(x=range(files), y=df['Alçada (m)'], color='blue', label='Dades_
↳ originals')
plt.fill_between(range(files), min(df['Alçada (m)']), max(df['Alçada (m)']),
↳ color='blue', alpha=0.2, label='Rang dades originals')

plt.fill_between(range(files), min(df_modificada['Alçada (m)']),
↳ max(df_modificada['Alçada (m)']), color='orange', alpha=0.2, label='Rang_
↳ dades originals')
files, columnes = df_modificada.shape
sns.scatterplot(x=range(files), y=df_modificada['Alçada (m)'], color='orange',
↳ label='Dades modificades')
plt.legend(loc='upper right')
plt.plot()
```

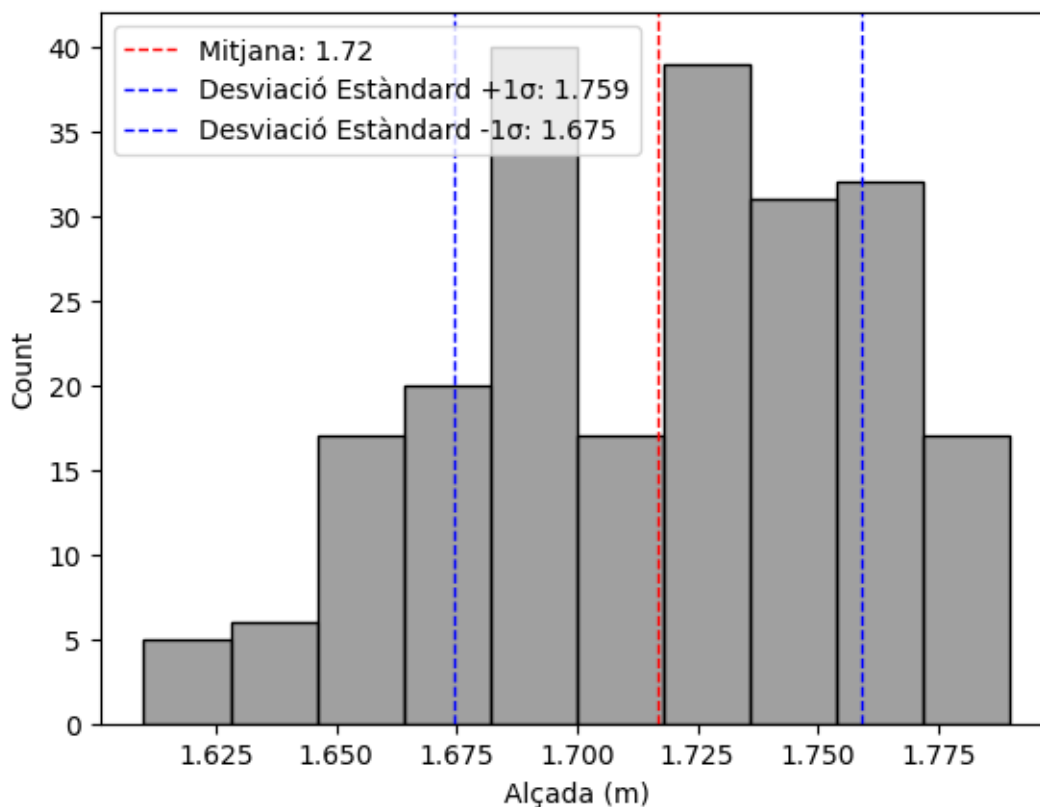
```
[ ]: [ ]
```



```
[ ]: variancia = df_modificada['Alçada (m)'].var()
mitjana = df_modificada['Alçada (m)'].mean()
desviacio_estandard = df_modificada['Alçada (m)'].std()
rang = df_modificada['Alçada (m)'].max() - df_modificada['Alçada (m)'].min()

sns.histplot(df_modificada['Alçada (m)'], bins=10, edgecolor='black',
             color='grey')
plt.axvline(mitjana, color='red', linestyle='dashed', linewidth=1,
            label=f'Mitjana: {mitjana:.2f}')

# Afegim la representació d'errors per a la desviació estàndard
plt.axvline(mitjana + desviacio_estandard, color='blue', linestyle='dashed',
            linewidth=1, label=f'Desviació Estàndard +1 : {mitjana + desviacio_estandard:.3f}')
plt.axvline(mitjana - desviacio_estandard, color='blue', linestyle='dashed',
            linewidth=1, label=f'Desviació Estàndard -1 : {mitjana - desviacio_estandard:.3f}')
plt.legend()
plt.show()
```

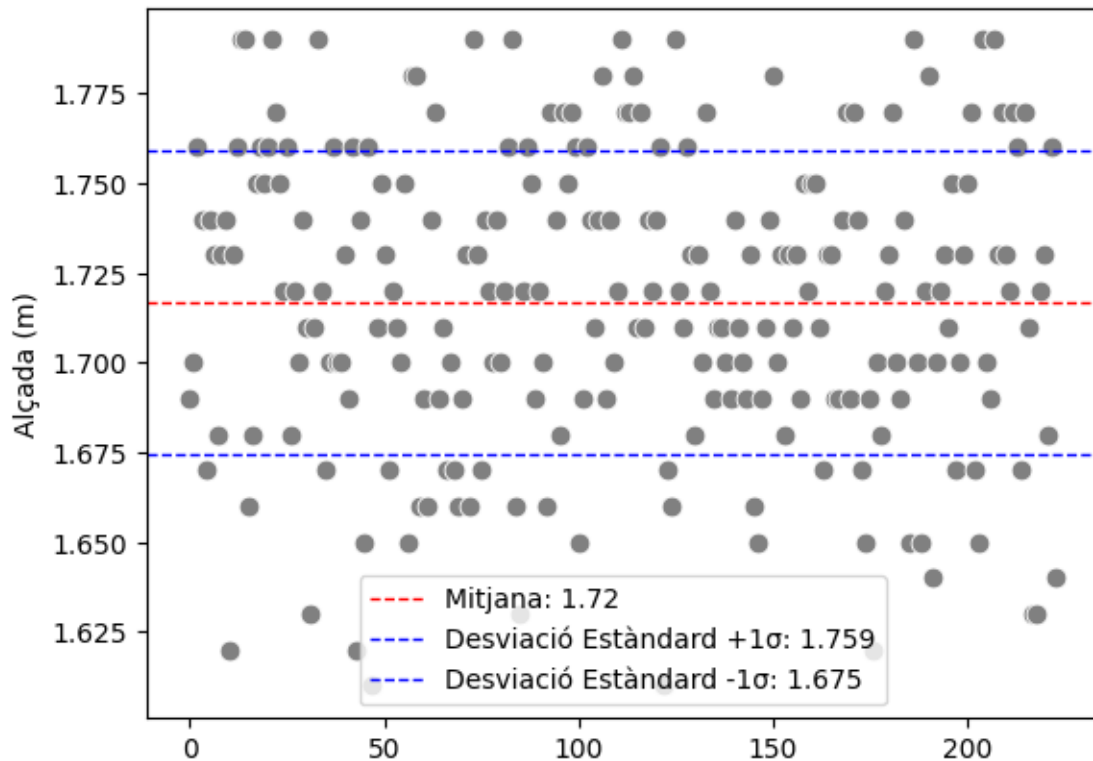


```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt

files, columnes = df_modificada.shape
sns.scatterplot(x=range(files), y=df_modificada['Alçada (m)'], color='grey',
               ↪s=60)

plt.axhline(mitjana, color='red', linestyle='dashed', linewidth=1,
           ↪label=f'Mitjana: {mitjana:.2f}')

# Afegim la representació d'errors per a la desviació estàndard
plt.axhline(mitjana + desviacio_estandard, color='blue', linestyle='dashed',
           ↪linewidth=1, label=f'Desviació Estàndard +1 : {mitjana + desviacio_estandard:.
           ↪3f}')
plt.axhline(mitjana - desviacio_estandard, color='blue', linestyle='dashed',
           ↪linewidth=1, label=f'Desviació Estàndard -1 : {mitjana - desviacio_estandard:.
           ↪3f}')
plt.legend()
plt.show()
```



```
[ ]: print(f"Mitjana: {mitjana:.3f}")
      print(f"Variança: {variancia:.3f}")
      print(f"Desviació Estàndard: {desviacio_estandard:.3f}")
      print(f"Rang: {rang:.3f}")
```

Mitjana: 1.717

Variança: 0.002

Desviació Estàndard: 0.042

Rang: 0.180

El càlcul d'aquests paràmetres ens permet:

- **Comparació entre conjunts de dades similars:** La variància ajuda a entendre com de variable és cada conjunt de dades. Això és útil quan vols saber si les dades són consistentes o si hi ha molta diversitat.
- **Representativitat de les mostres:** si el conjunt de dades té una alta desviació estàndard, pot indicar que la mostra no és representativa de la població general i podria provocar que les conclusions extremes no siguin fiables.
- **Detecció d'anomalies:** Les dades amb variabilitat extremadament alta i/o que es troben a més de dues o tres desviacions estàndard de la mitjana es consideren anomalies o valors extrems (podrien ser errors o casos que necessiten un estudi especial).

Percentils Els percentils són mesures estadístiques que divideixen un conjunt de dades ordenades en cent parts iguals, permetent entendre millor la distribució dels valors. Els percentils més utilitzats en la ciència de dades són:

- **Percentil 25 (Primer Quartil, Q1):** Indica que el 25% de les dades estan per sota d'aquest valor. És útil per identificar el valor per sota del qual es troba el quart inferior de les dades.
- **Percentil 50 (Mediana, Q2):** És el valor central de les dades. El 50% de les dades estan per sota d'aquest valor i el 50% per sobre. Sovint es considera un indicador del “valor mitjà” d'un conjunt de dades.
- **Percentil 75 (Tercer Quartil, Q3):** Indica que el 75% de les dades estan per sota d'aquest valor. Ajuda a identificar el límit superior de la meitat inferior de les dades.
- **Percentil 90:** Mostra el valor per sota del qual es troba el 90% de les dades. És útil per identificar valors alts en una distribució de dades.
- **Percentil 95:** Similar al percentil 90, però indica que el 95% de les dades estan per sota d'aquest valor. S'utilitza per detectar possibles valors extrems o outliers.
- **Percentil 99:** Mostra el valor per sota del qual es troba el 99% de les dades. Es fa servir per identificar els valors més extrems dins d'un conjunt de dades.

```
[ ]: percentil_25 = df['Hores d\Estudi'].quantile(0.25)
print(f'Percentil 25 : {int(percentil_25)}')
percentil_50 = df['Hores d\Estudi'].quantile(0.50)
print(f'Percentil 50 : {int(percentil_50)}')
percentil_75 = df['Hores d\Estudi'].quantile(0.75)
print(f'Percentil 75 : {int(percentil_75)}')
percentil_90 = df['Hores d\Estudi'].quantile(0.90)
print(f'Percentil 90 : {int(percentil_90)}')
percentil_95 = df['Hores d\Estudi'].quantile(0.95)
print(f'Percentil 95 : {int(percentil_95)}')
percentil_99 = df['Hores d\Estudi'].quantile(0.99)
print(f'Percentil 99 : {int(percentil_99)}')
```

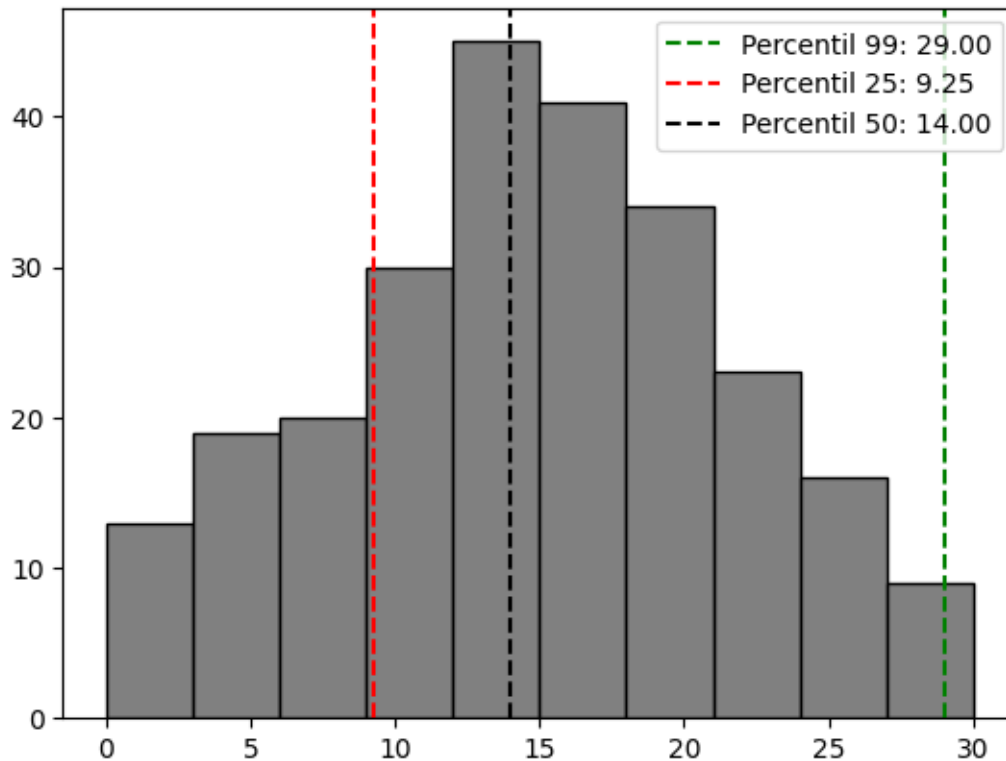
```
Percentil 25 : 9
Percentil 50 : 14
Percentil 75 : 19
Percentil 90 : 23
Percentil 95 : 26
Percentil 99 : 29
```

```
[ ]: plt.hist(df['Hores d\Estudi'],color='gray', edgecolor='black')

plt.axvline(percentil_99, color='green', linestyle='--', label=f'Percentil 99:␣
↪{percentil_99:.2f}')
plt.axvline(percentil_25, color='red', linestyle='--', label=f'Percentil 25:␣
↪{percentil_25:.2f}')
plt.axvline(percentil_50, color='black', linestyle='--', label=f'Percentil 50:␣
↪{percentil_50:.2f}')
```

```
plt.legend()
plt.plot()
```

```
[ ]: [ ]
```



2 Com s'han generat les dades

```
[ ]: !pip install faker
```

Collecting faker

Downloading faker-38.2.0-py3-none-any.whl.metadata (16 kB)

Requirement already satisfied: tzdata in /usr/local/lib/python3.12/dist-packages (from faker) (2025.2)

Downloading faker-38.2.0-py3-none-any.whl (2.0 MB)

2.0/2.0 MB

23.7 MB/s eta 0:00:00

Installing collected packages: faker

Successfully installed faker-38.2.0

```
[ ]: #Funcio per generar noms aleatoris
from faker import Faker
import numpy as np

def name_generator(prob_M,prob_F,idioma= 'es'):
    fake = Faker(idioma)

    if np.random.random() < prob_M:
        return f'{fake.first_name_male()} {fake.last_name()}'
    else:
        return f'{fake.first_name_female()} {fake.last_name()}'

[ ]: name_generator(prob_M=0.5,prob_F=0.5)

[ ]: 'Dan Roman'

[ ]: [name_generator(prob_M=0.5,prob_F=0.5) for item in range(10)]

[ ]: ['Luna Tirado',
      'Ernesto Vila',
      'Cirino Fonseca',
      'Jose Ramón Peiró',
      'María Jesús Lloret',
      'Julián Ripoll',
      'Rogelio Villaverde',
      'Eligia Plaza',
      'Marcos Carbajo',
      'Iker Cañete']

[ ]: import pandas as pd
import numpy as np

np.random.seed(42) #establir "seed" (llavor)
n = 250 #nombre de registres desitjat

# Generar dades per l'edat amb la distribució desitjada (la majoria entre 18 i
↳25)
edat = np.random.choice([18, 19, 20, 21, 22, 23, 24, 25, 32, 44, 36, 29],
↳size=n, p=[0.2, 0.2, 0.1, 0.09, 0.08, 0.08, 0.075, 0.075, 0.03,0.03,0.02,0.
↳02])

# Generar alçada correlacionada amb l'edat (-0.3)
alçada = 1.72 + -0.003 * (edat - np.mean(edat)) + np.random.normal(0, 0.05, n)
alçada = np.clip(alçada, 1.59, 2.10) # Limitar les alçades al rang 1.59-2.10
↳metres

# Programació i Bases de Dades (correlació de 0.7)
```



```

programacio = np.random.normal(7, 1, n)
bases_dades = 0.7 * programacio + np.random.normal(0, 3, n)
bases_dades = np.clip(bases_dades, 0.00,10.00)

# Hores d'estudi correlacionades amb programació (0.5) i sistemes operatius (0.
↳2)
hores_estudi = np.round(0.5 * programacio + np.random.normal(10,7, n)).
↳astype(int)
hores_estudi = np.clip(hores_estudi, 0,30)

# Sistemes operatius correlacionats amb hores d'estudi (0.2)
sistemes_operatius = 0.2 * hores_estudi + np.random.normal(5, 4, n)
sistemes_operatius = np.clip(sistemes_operatius, 0.00, 10.00)

# Desenvolupament Web correlacionat amb Programació (0.6) i Bases de Dades (0.3)
desenvolupament_web = 0.6 * programacio + 0.3 * bases_dades + np.random.
↳normal(1, 2, n)
desenvolupament_web = np.clip(desenvolupament_web, 0.00, 10.00)
#Faltes d'Assistència correlacionades amb l'edat (0.5)
faltes_assistencia = np.round(0.5 * edat + np.random.normal(-10, 15, n)).
↳astype(int)
faltes_assistencia = np.clip(faltes_assistencia, 0, 25)

# Generem l'id d'alumne i els noms fictisis
ids = [f'ALU-{i:04d}' for i in np.arange(1000, 1000 + n)]
noms = [name_generator(prob_M=0.8,prob_F=0.2) for item in range(250)]

# Crear DataFrame
df = pd.DataFrame({
    'ID Alumne': ids,
    'Nom': noms,
    'Edat': np.round(edat),
    'Alçada (m)': np.round(alçada, 2),
    'Programació': np.round(programacio, 1),
    'Bases de Dades': np.round(bases_dades, 1),
    'Sistemes Operatius': np.round(sistemes_operatius, 1),
    'Desenvolupament Web': np.round(desenvolupament_web, 1),
    'Hores d\'Estudi': np.round(hores_estudi, 1),
    'Faltes d\'Assistència': np.round(faltes_assistencia, 1)
})

# Guardar el DataFrame en un fitxer CSV
#df.to_csv('alumnes_dades_correlacionades.csv', index=False)

```